Race Conditions in Verilog

October 12, 2026

1 Introduction

A **race condition** occurs when the result of a Verilog simulation depends on the execution order of concurrent processes, and that order is not guaranteed.

Race conditions commonly occur when multiple `always` or `initial` blocks access or modify the same variable without proper synchronization.

2 Illustrative Example

```
module race_demo;

    reg clk = 0;
    reg q = 0;

    always #5 clk = ~clk;

    // Design logic
    always @(posedge clk)
        q = 1'b1;

    // Testbench
    initial begin
        @(posedge clk);
        $display("q = %b", q);
    end

endmodule
```

2.1 Explanation

At the rising clock edge, both the design block and the testbench execute in the Active region.

- If the design executes first, the output is `q = 1`.
- If the testbench executes first, the output is `q = 0`.

The output depends on the execution order. This is a race condition.

3 Avoiding Race Conditions

Use nonblocking assignments (<=) for clocked sequential logic.

```
always @(posedge clk)
    q <= 1'b1;
```

The update is scheduled for the NBA (Nonblocking Assignment) region.

To sample the updated value, the testbench can use:

```
initial begin
    @(posedge clk);
    #1;
    $display("q = %b", q);
end
```

The output is `q = 1`, assuming the delay advances to a later simulation time.

4 Blocking vs. Nonblocking Assignments

Blocking (=)	**Nonblocking (<=)**
Updates immediately when executed	Schedules the update for the NBA region
Commonly used in combinational logic	Commonly used in sequential logic
Can cause races in concurrent code	Helps model simultaneous register updates

5 Prevention Guidelines

- Use nonblocking assignments for clocked sequential logic.
- Use blocking assignments for combinational procedural logic.
- Avoid multiple procedural blocks writing to the same variable.
- Synchronize testbench stimulus and sampling.
- Never depend on the execution order of independent concurrent processes.

6 Summary

- A race condition occurs when simulation results depend on an unspecified execution order.
- Concurrent processes may execute in different orders within the Active region.
- Nonblocking assignments help model simultaneous register updates.
- Proper coding styles and synchronization help prevent race conditions.